

TP 5 — Kafka Streams, Airflow Avancé & Observabilité

Cours 5 | Développer pour le Cloud | YNOV Campus Montpellier — Master 2

Date : 08/06/2026 | Durée TP : 3h30 | Plateforme : Google Cloud Platform

Contexte entreprise — FraudGuard

FraudGuard est la néobanque numérique du groupe Meridian. Elle traite **3 millions de transactions par jour** (35 transactions/seconde en moyenne, pics à 200/s les vendredis soir et les jours de paie). Fin 2025, elle a subi une fraude de 1,2M€ : des scripts automatisés effectuaient des centaines de micro-virements de 0,50€ vers des comptes mules, indétectables par les règles statiques. La banque de France a émis une injonction : FraudGuard doit implémenter un système de détection de fraude en temps réel sous 6 mois. La solution retenue : **Kafka Streams** pour la détection temps réel (analyse de patterns sur des fenêtres de 5 minutes) + **Apache Airflow** pour les rapports réglementaires quotidiens et le réentraînement des modèles ML. Vous êtes le/la Streaming & Data Engineer responsable de ce projet.

Prérequis validés (Cours 4) :

- Apache Airflow déployé sur GKE avec Helm
- Kafka Strimzi opérationnel (TP3)
- DAGs Airflow de base maîtrisés

Objectifs de ce TP :

- Implémenter un pipeline Kafka Streams pour la détection de fraude en temps réel
- Maîtriser le windowing Kafka Streams (Tumbling Windows, Sliding Windows)
- Créer des DAGs Airflow avancés : TriggerDagRunOperator, Dynamic Task Mapping, KubernetesPodOperator
- Déployer une stack d'observabilité complète (Prometheus + Grafana + alerting)
- Connecter les insights Kafka Streams aux rapports Airflow (pipeline hybride temps réel + batch)

Livrables attendus :

- ☐ Pipeline Kafka Streams qui détecte les micro-transactions suspectes en temps réel
- ☐ Alertes de fraude générées et consommées par un service d'alerte
- ☐ DAG Airflow de rapport de fraude quotidien avec Dynamic Task Mapping
- ☐ Dashboard Grafana avec métriques Kafka Streams en temps réel
- ☐ README.md décrivant l'architecture de détection de fraude complète

Partie 1 — Kafka Streams : Détection de fraude en temps réel

Kafka Streams est une bibliothèque Java/Scala de traitement de flux qui s'exécute dans votre application (pas un cluster séparé). Elle lit depuis des topics Kafka, applique des transformations et des agrégations, et écrit les résultats dans d'autres topics. Pour FraudGuard, on utilisera une implémentation Node.js via le package `kafka-streams`.

1.1 — Déployer le cluster Kafka FraudGuard

```
# Namespace dédié FraudGuard
kubectl create namespace fraudguard

# Déployer le cluster Kafka avec Strimzi (si pas encore fait)
kubectl apply -f https://strimzi.io/install/latest?namespace=kafka -n kafka
kubectl wait --for=condition=Ready pod -l name=strimzi-cluster-operator -n kafka --
timeout=120s
```

Créez `kafka/fraudguard-cluster.yaml` :

```
apiVersion: kafka.strimzi.io/v1beta2
kind: Kafka
metadata:
  name: fraudguard-kafka
  namespace: fraudguard
spec:
  kafka:
    version: 3.7.0
    replicas: _____ # 3 brokers
    listeners:
      - name: plain
        port: 9092
        type: internal
        tls: false
    config:
      num.partitions: 6
      default.replication.factor: 3
      min.insync.replicas: 2
      # Rétention courte : les transactions sont analysées dans les 24h
      log.retention.hours: _____ # 24
      # Clé de compression pour réduire la bande passante
      compression.type: snappy
    storage:
      type: persistent-claim
      size: 10Gi
      deleteClaim: true
    zookeeper:
      replicas: 0
    entityOperator:
      topicOperator: {}
      userOperator: {}
---
# Topics FraudGuard
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaTopic
metadata:
  name: transactions-raw
  namespace: fraudguard
```

```

  labels:
    strimzi.io/cluster: fraudguard-kafka
spec:
  partitions: 6
  replicas: 3
  config:
    retention.ms: "86400000" # 24h
---
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaTopic
metadata:
  name: fraud-alerts
  namespace: fraudguard
  labels:
    strimzi.io/cluster: fraudguard-kafka
spec:
  partitions: 3
  replicas: 3
  config:
    # Les alertes sont critiques : rétention 30 jours pour audit
    retention.ms: "2592000000" # "2592000000" (30 jours)
---
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaTopic
metadata:
  name: transaction-aggregates
  namespace: fraudguard
  labels:
    strimzi.io/cluster: fraudguard-kafka
spec:
  partitions: 6
  replicas: 3
  config:
    # Topic compacté : garder la dernière valeur par clé (compte_id)
    cleanup.policy: "compact" # compact
    retention.ms: "604800000"

```

```
kubectl apply -f kafka/fraudguard-cluster.yaml
```

```

# Attendre que le cluster soit prêt
kubectl wait kafka/fraudguard-kafka \
  --for=condition=Ready \
  --timeout=300s \
  -n fraudguard

```

1.2 — Simulateur de transactions : le Producer

Créez `fraud-detection/producer/transaction-producer.js` :

```

/**
 * FraudGuard Transaction Producer
 * Simule le flux de transactions de la néobanque Meridian.
 * Inclut des transactions légitimes ET des patterns frauduleux.
 */
const { Kafka, Partitioners } = require('kafkajs');

const kafka = new Kafka({
  clientId: 'fraudguard-tx-producer',
  brokers: [process.env.KAFKA_BOOTSTRAP_SERVERS || 'localhost:9092'],
});

const producer = kafka.producer({
  createPartitioner: Partitioners.LegacyPartitioner,
});

// Comptes légitimes FraudGuard
const LEGITIMATE_ACCOUNTS = [
  { id: 'ACC-1001', name: 'Alice Martin', avg_tx: 85, daily_limit: 2000 },
  { id: 'ACC-1002', name: 'Bob Dupont', avg_tx: 120, daily_limit: 5000 },
  { id: 'ACC-1003', name: 'Claire Leroy', avg_tx: 45, daily_limit: 1500 },
  { id: 'ACC-1004', name: 'David Moreau', avg_tx: 200, daily_limit: 10000 },
];

// Compte frauduleux (micro-transactions répétées → détection par Kafka Streams)
const FRAUD_ACCOUNT = { id: 'ACC-9999', name: 'Compte Mule', avg_tx: 0.50 };

let txCounter = 0;

function generateTransaction(account, isFraud = false) {
  txCounter++;
  const amount = isFraud
    ? 0.50 + Math.random() * 0.50 // Micro-transaction 0.50-1.00€
    : account.avg_tx * (0.5 + Math.random()); // Transaction normale

  return {
    tx_id: `TX-${Date.now()}-${txCounter.toString().padStart(6, '0')}`,
    account_id: account.id,
    account_name: account.name,
    amount: parseFloat(amount.toFixed(2)),
    currency: 'EUR',
    merchant: isFraud ? 'MERCHANT-MULE-001' : `MERCHANT-${Math.floor(Math.random() * 50) + 1}`,
    tx_type: isFraud ? 'TRANSFER' : (Math.random() > 0.3 ? 'PAYMENT' : 'WITHDRAWAL'),
    ip_address: isFraud ? '185.234.219.45' : `192.168.${Math.floor(Math.random() * 255)}${Math.floor(Math.random() * 255)}`,
    device_fingerprint: isFraud ? 'BOT-FINGERPRINT-XYZ' : `DEVICE-${account.id}`,
    timestamp: new Date().toISOString(),
    is_fraud_simulation: isFraud, // Champ pour évaluer la détection
  };
}

```

```

}

async function startProducing() {
  await producer.connect();
  console.log('[FraudGuard] Producer démarré – simulation du flux transactionnel');

  // Transactions légitimes : une toutes les 200ms (5/seconde × 4 comptes = 20/s)
  setInterval(async () => {
    const account = LEGITIMATE_ACCOUNTS[Math.floor(Math.random() *
LEGITIMATE_ACCOUNTS.length)];
    const tx = generateTransaction(account, false);

    await producer.send({
      topic: '_____', // 'transactions-raw'
      messages: [{
        key: tx.account_id, // Clé = compte → même partition pour un compte
        value: JSON.stringify(tx),
        headers: { 'tx-type': tx.tx_type },
      }],
    });
  }, 200);

  // Simulation d'attaque par micro-transactions : rafale de 20 transactions en 30s
  // Pattern frauduleux : beaucoup de petites transactions depuis le même compte
  setInterval(async () => {
    console.log('[SIMULATION] Lancement d\'une attaque par micro-transactions...');
    for (let i = 0; i < 25; i++) {
      const tx = generateTransaction(FRAUD_ACCOUNT, true);
      await producer.send({
        topic: 'transactions-raw',
        messages: [{
          key: FRAUD_ACCOUNT.id,
          value: JSON.stringify(tx),
        }],
      });
      await new Promise(r => setTimeout(r, _____)); // 500 (délai entre micro-
transactions)
    }
    console.log('[SIMULATION] Attaque par micro-transactions terminée');
  }, 60000); // Toutes les 60 secondes
}

process.on('SIGTERM', async () => {
  await producer.disconnect();
  process.exit(0);
});

startProducing().catch(console.error);

```

1.3 — Kafka Streams : Détection par fenêtres temporelles

Kafka Streams permet d'analyser des flux de données avec des **fenêtres temporelles**. Une **Tumbling Window** de 5 minutes agrège toutes les transactions d'un compte sur 5 minutes consécutives et non chevauchantes. Si un compte fait > 10 transactions de < 2€ dans une fenêtre de 5 minutes → alerte de fraude.

Créez `fraud-detection/streams/fraud-detector.js` :

```
/**
 * FraudGuard Fraud Detector – Kafka Streams en Node.js
 * Détecte 3 patterns de fraude :
 * 1. Micro-transactions répétées (> 10 tx < 2€ en 5 minutes)
 * 2. Vitesse élevée (> 20 transactions en 5 minutes, tous montants)
 * 3. Montant anormalement élevé (> 5x la moyenne du compte sur 30 jours)
 */
const { Kafka } = require('kafkajs');

const kafka = new Kafka({
  clientId: 'fraudguard-streams-detector',
  brokers: [process.env.KAFKA_BOOTSTRAP_SERVERS || 'localhost:9092'],
});

const consumer = kafka.consumer({ groupId: 'fraud-detection-group' });
const producer = kafka.producer();

// =====
// État en mémoire (en prod : utiliser un State Store Redis ou RocksDB)
// Fenêtre glissante de 5 minutes par compte
// =====
const WINDOW_SIZE_MS = 5 * 60 * 1000 // 5 minutes en ms
const MICRO_TX_THRESHOLD_AMOUNT = 2.00; // < 2€ = micro-transaction
const MICRO_TX_THRESHOLD_COUNT = 10; // > 10 micro-tx en 5 min = fraude
const VELOCITY_THRESHOLD = 20; // > 20 tx en 5 min = fraude

// Map : account_id → liste de transactions dans la fenêtre courante
const windowedTransactions = new Map();

function pruneExpiredTransactions(accountId) {
  const now = Date.now();
  const txList = windowedTransactions.get(accountId) || [];
  const fresh = txList.filter(tx => now - tx._received_at < WINDOW_SIZE_MS);
  windowedTransactions.set(accountId, fresh);
  return fresh;
}

async function analyzeTransaction(transaction) {
  const accountId = transaction.account_id;
  const now = Date.now();

  // Ajouter la transaction à la fenêtre en mémoire
  const txList = pruneExpiredTransactions(accountId);
  txList.push({ ...transaction, _received_at: now });
  windowedTransactions.set(accountId, txList);
}
```

```

const alerts = [];

// ---- Pattern 1 : Micro-transactions répétées ----
const microTxCount = txList.filter(tx => tx.amount < _____).length; //
MICRO_TX_THRESHOLD_AMOUNT
if (microTxCount >= MICRO_TX_THRESHOLD_COUNT) {
  alerts.push({
    alert_type: 'MICRO_TRANSACTION_PATTERN',
    severity: 'HIGH',
    description: `${microTxCount} micro-transactions (<
${MICRO_TX_THRESHOLD_AMOUNT}€) en 5 minutes`,
    micro_tx_count: microTxCount,
    total_amount: txList
      .filter(tx => tx.amount < MICRO_TX_THRESHOLD_AMOUNT)
      .reduce((sum, tx) => sum + tx.amount, 0)
      .toFixed(2),
  });
}

// ---- Pattern 2 : Vitesse élevée ----
if (txList.length >= _____) { // VELOCITY_THRESHOLD
  alerts.push({
    alert_type: 'HIGH_VELOCITY',
    severity: 'CRITICAL',
    description: `${txList.length} transactions en 5 minutes – vitesse anormale`,
    tx_count_5min: txList.length,
  });
}

// ---- Pattern 3 : IP suspecte (adresse connue de bots) ----
const suspiciousIPs = ['185.234.219.45', '31.220.0.0/24'];
if (suspiciousIPs.includes(transaction.ip_address)) {
  alerts.push({
    alert_type: 'SUSPICIOUS_IP',
    severity: 'MEDIUM',
    description: `Adresse IP suspecte détectée : ${transaction.ip_address}`,
  });
}

return alerts;
}

async function publishAlert(transaction, alert) {
  const fraudAlert = {
    alert_id: `ALERT-${Date.now()}-${transaction.account_id}`,
    account_id: transaction.account_id,
    account_name: transaction.account_name,
    triggering_tx_id: transaction.tx_id,
    triggering_amount: transaction.amount,
    ...alert,
    window_size_minutes: WINDOW_SIZE_MS / 60000,
  };

```

```

    detected_at: new Date().toISOString(),
    action_recommended: alert.severity === 'CRITICAL' ? 'BLOCK_ACCOUNT' : 'REVIEW',
  };

  await producer.send({
    topic: '_____', // 'fraud-alerts'
    messages: [{
      key: transaction.account_id,
      value: JSON.stringify(fraudAlert),
    }],
  });

  console.log(`[ALERTE ${alert.severity}] ${alert.alert_type} - Compte
${transaction.account_id}`);
  console.log(` → ${alert.description}`);
}

async function startDetection() {
  await Promise.all([consumer.connect(), producer.connect()]);
  console.log('[FraudGuard] Moteur de détection Kafka Streams démarré');

  await consumer.subscribe({ topics: ['transactions-raw'], fromBeginning: false });

  // Stats de monitoring
  let txProcessed = 0;
  let alertsGenerated = 0;

  setInterval(() => {
    const accountsMonitored = windowedTransactions.size;
    const totalWindowedTx = Array.from(windowedTransactions.values())
      .reduce((sum, list) => sum + list.length, 0);
    console.log(`[STATS] Traitées: ${txProcessed} tx | Alertes: ${alertsGenerated} |
Comptes moniteurs: ${accountsMonitored} | En fenêtre: ${totalWindowedTx} tx`);
  }, 30000);

  await consumer.run({
    eachMessage: async ({ message }) => {
      const transaction = JSON.parse(message.value.toString());
      txProcessed++;

      const alerts = await analyzeTransaction(transaction);

      for (const alert of alerts) {
        await publishAlert(transaction, alert);
        alertsGenerated++;
      }
    },
  });
}

process.on('SIGTERM', async () => {
  await Promise.all([consumer.disconnect(), producer.disconnect()]);

```



```
    process.exit(0);
  });

  startDetection().catch(console.error);
```

Question : Le moteur de détection stocke l'état des fenêtres en mémoire locale (`windowedTransactions`). Quels sont les deux problèmes critiques de cette approche si on déploie 3 réplicas du `fraud-detector` sur GKE ? Quelle solution de State Store recommanderiez-vous ?

Réponse :

1.4 — Service d'alerte : consommer et agir sur les fraudes

Créez `fraud-detection/alert-service/alert-handler.js` :

```
/**
 * FraudGuard Alert Handler
 * Consomme les alertes Kafka et déclenche les actions :
 * - Bloquer le compte (sévérité CRITICAL)
 * - Notifier le Risk Manager (sévérité HIGH)
 * - Enregistrer pour audit (toutes les alertes)
 */
const { Kafka } = require('kafkajs');
const { Firestore } = require('@google-cloud/firestore');

const kafka = new Kafka({
  clientId: 'fraudguard-alert-handler',
  brokers: [process.env.KAFKA_BOOTSTRAP_SERVERS || 'localhost:9092'],
});

const consumer = kafka.consumer({ groupId: '_____ ' }); // 'alert-handler-group'
const db = new Firestore({ projectId: process.env.GCP_PROJECT });

async function processAlert(alert) {
  console.log(`[ALERT HANDLER] Traitement : ${alert.alert_type} - ${alert.account_id}`);

  // 1. Enregistrer l'alerte dans Firestore pour audit
  await db.collection('fraud_alerts').doc(alert.alert_id).set({
    ...alert,
    processed_at: Firestore.Timestamp.now(),
    status: 'PENDING_REVIEW',
  });

  // 2. Action selon la sévérité
  if (alert.severity === 'CRITICAL') {
    // Bloquer le compte immédiatement
    await blockAccount(alert.account_id, alert.alert_id);
    await notifyRiskManager(alert, 'URGENT: Compte bloqué automatiquement');
  } else if (alert.severity === '_____') { // 'HIGH'
```

```

    // Notifier le Risk Manager pour revue manuelle
    await notifyRiskManager(alert, 'Action requise : pattern de fraude détecté');
    // Limiter les transactions (au lieu de bloquer)
    await limitAccountTransactions(alert.account_id, 50); // Max 50€ par
transaction

  } else if (alert.severity === 'MEDIUM') {
    // Enregistrer pour analyse batch par Airflow
    console.log(`[AUDIT] Alerte MEDIUM enregistrée pour revue Airflow quotidienne`);
  }
}

async function blockAccount(accountId, alertId) {
  // En production : appel à l'API Core Banking
  console.log(`[ACTION] BLOCAGE du compte ${accountId} – Alerte ${alertId}`);
  await db.collection('blocked_accounts').doc(accountId).set({
    blocked_at: Firestore.Timestamp.now(),
    reason: alertId,
    status: 'BLOCKED',
  });
}

async function limitAccountTransactions(accountId, maxAmount) {
  console.log(`[ACTION] Limitation du compte ${accountId} à ${maxAmount}
€/transaction`);
}

async function notifyRiskManager(alert, message) {
  console.log(`[NOTIFICATION] Risk Manager : ${message}`);
  console.log(`  Compte: ${alert.account_id} | Type: ${alert.alert_type}`);
  // En prod : email/SMS/Slack via API
}

async function startAlertHandling() {
  await consumer.connect();
  await consumer.subscribe({ topics: ['fraud-alerts'], fromBeginning: false });

  await consumer.run({
    eachMessage: async ({ message }) => {
      const alert = JSON.parse(message.value.toString());
      await processAlert(alert);
    },
  });
}

process.on('SIGTERM', async () => { await consumer.disconnect(); process.exit(0);
});
startAlertHandling().catch(console.error);

```

1.5 — Déployer le pipeline de détection sur GKE

```
PROJECT_ID=$(gcloud config get-value project)

# Builder toutes les images
for service in producer streams alert-service; do
  docker build \
    -t europe-west9-docker.pkg.dev/${PROJECT_ID}/tp2-
registry/fraudguard-${service}:v1 \
    fraud-detection/${service}/
  docker push europe-west9-docker.pkg.dev/${PROJECT_ID}/tp2-
registry/fraudguard-${service}:v1
done
```

Créez `k8s/fraudguard-deployments.yaml` :

```
# Transaction Producer
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tx-producer
  namespace: fraudguard
spec:
  replicas: 1
  selector:
    matchLabels:
      app: tx-producer
  template:
    metadata:
      labels:
        app: tx-producer
    spec:
      containers:
        - name: tx-producer
          image: europe-west9-docker.pkg.dev/[PROJECT_ID]/tp2-registry/fraudguard-
producer:v1
          env:
            - name: KAFKA_BOOTSTRAP_SERVERS
              value: "fraudguard-kafka-kafka-
bootstrap.fraudguard.svc.cluster.local:9092"
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
---
# Fraud Detector (Kafka Streams)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fraud-detector
  namespace: fraudguard
spec:
  # IMPORTANT : replicas à 1 pour éviter le problème d'état distribué (voir question
```

1.3)

```
replicas: _____ # 1
selector:
  matchLabels:
    app: fraud-detector
template:
  metadata:
    labels:
      app: fraud-detector
  spec:
    containers:
      - name: fraud-detector
        image: europe-west9-docker.pkg.dev/[PROJECT_ID]/tp2-registry/fraudguard-
streams:v1
        env:
          - name: KAFKA_BOOTSTRAP_SERVERS
            value: "fraudguard-kafka-kafka-
bootstrap.fraudguard.svc.cluster.local:9092"
        resources:
          requests:
            cpu: "500m"
            memory: "512Mi"
          limits:
            cpu: "2"
            memory: "2Gi"
---
# Alert Handler
apiVersion: apps/v1
kind: Deployment
metadata:
  name: alert-handler
  namespace: fraudguard
spec:
  replicas: 2
  selector:
    matchLabels:
      app: alert-handler
  template:
    metadata:
      labels:
        app: alert-handler
    spec:
      containers:
        - name: alert-handler
          image: europe-west9-docker.pkg.dev/[PROJECT_ID]/tp2-registry/fraudguard-
alert-service:v1
          env:
            - name: KAFKA_BOOTSTRAP_SERVERS
              value: "fraudguard-kafka-kafka-
bootstrap.fraudguard.svc.cluster.local:9092"
            - name: GCP_PROJECT
              value: "[PROJECT_ID]"
```

```
resources:
  requests:
    cpu: "200m"
    memory: "256Mi"
```

```
kubectl apply -f k8s/fraudguard-deployments.yaml

# Observer le pipeline en temps réel (3 fenêtres de logs simultanées)
# Terminal 1 : Producer
kubectl logs -f deployment/tx-producer -n fraudguard

# Terminal 2 : Fraud Detector
kubectl logs -f deployment/fraud-detector -n fraudguard

# Terminal 3 : Alert Handler
kubectl logs -f deployment/alert-handler -n fraudguard

# Vérifier les alertes dans Firestore
gcloud firestore databases list
# → Console GCP → Firestore → Collection "fraud_alerts"
```

Partie 2 — Airflow Avancé : Rapports de Fraude et Réentraînement ML

Les alertes Kafka sont du temps réel. Mais les régulateurs demandent des rapports quotidiens consolidés : taux de fraude par type, par heure, par compte — avec des analyses que le streaming seul ne peut pas faire efficacement. Airflow orchestre ces traitements batch post-hoc.

2.1 — Dynamic Task Mapping : rapport par type d'alerte

Le **Dynamic Task Mapping** (Airflow 2.3+) génère dynamiquement des tâches en parallèle à partir d'une liste calculée au runtime — sans connaître à l'avance le nombre de tâches.

Créez `dags/fraud_daily_report.py` :

```
"""
DAG : FraudGuard Daily Fraud Report
Génère un rapport quotidien de fraude pour chaque type d'alerte.
Utilise le Dynamic Task Mapping pour paralléliser l'analyse par type.
"""

from __future__ import annotations

from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.google.cloud.operators.bigquery import
BigQueryInsertJobOperator
from airflow.providers.google.cloud.operators.kubernetes_engine import
GKEStartPodOperator

ALERT_TYPES = [
```

```

'MICRO_TRANSACTION_PATTERN',
'HIGH_VELOCITY',
'SUSPICIOUS_IP',
]

with DAG(
    dag_id='fraudguard_daily_report',
    description='Rapport quotidien de fraude FraudGuard – MIF II & ACPR',
    default_args={
        'retries': 2,
        'retry_delay': timedelta(minutes=5),
        'email_on_failure': True,
        'email': ['fraud-ops@fraudguard.fr'],
    },
    start_date=datetime(2026, 1, 1),
    schedule='0 7 * * *', # 07h00 chaque matin
    catchup=False,
    tags=['fraud', 'reporting', 'acpr'],
) as dag:

    # =====
    # Tâche 1 : Récupérer la liste des types d'alertes depuis Firestore
    # (En prod : requête Firestore pour les types actifs du jour)
    # =====
    def get_alert_types_for_date(**context):
        """Retourne la liste des types d'alertes à analyser pour hier."""
        execution_date = context['ds']
        print(f"[FraudGuard] Récupération des types d'alertes pour {execution_date}")
        # En prod : requête Firestore/BigQuery pour obtenir les types actifs
        return ALERT_TYPES

    fetch_alert_types = PythonOperator(
        task_id='fetch_alert_types',
        python_callable=get_alert_types_for_date,
    )

    # =====
    # Tâche 2 : Analyser chaque type d'alerte EN PARALLÈLE
    # Dynamic Task Mapping : une tâche par type d'alerte
    # =====
    def analyze_alert_type(alert_type: str, **context):
        """Analyser les statistiques d'un type d'alerte spécifique."""
        execution_date = context['ds']
        print(f"[FraudGuard] Analyse {alert_type} pour {execution_date}")

        # Simulation de l'analyse (en prod : requête BigQuery)
        import random
        stats = {
            'alert_type': alert_type,
            'date': execution_date,
            'count': random.randint(5, 200),

```

```

        'unique_accounts_affected': random.randint(2, 50),
        'total_amount_at_risk': round(random.uniform(100, 50000), 2),
        'avg_detection_latency_ms': random.randint(50, 500),
        'false_positive_rate': round(random.uniform(0.02, 0.15), 3),
    }

    print(f"[FraudGuard] {alert_type}: {stats['count']} alertes, "
          f"{stats['unique_accounts_affected']} comptes, "
          f"{stats['total_amount_at_risk']}€ à risque")
    return stats

# Dynamic Task Mapping : .expand() génère N tâches en parallèle
analyze_by_type = PythonOperator.partial(
    task_id='analyze_alert_type',
    python_callable=analyze_alert_type,
).expand(
    op_kwargs=[{'alert_type': t} for t in _____] # ALERT_TYPES
)

# =====
# Tâche 3 : Consolider les analyses de tous les types
# =====
def consolidate_report(**context):
    """Agréger les analyses de tous les types d'alertes en un rapport global."""
    # Récupérer les résultats de toutes les tâches expand
    all_stats = context['ti'].xcom_pull(task_ids='analyze_alert_type')

    if not all_stats:
        print("[FraudGuard] Aucune donnée à consolider")
        return

    total_alerts = sum(s['count'] for s in all_stats)
    total_amount = sum(s['total_amount_at_risk'] for s in all_stats)

    report = {
        'date': context['ds'],
        'total_alerts': total_alerts,
        'total_amount_at_risk': round(total_amount, 2),
        'breakdown_by_type': all_stats,
        'generated_at': datetime.now().isoformat(),
        'regulatory_compliant': True,
    }

    print(f"\n{'='*50}")
    print(f"RAPPORT FRAUDE FraudGuard - {context['ds']}")
    print(f"Total alertes : {total_alerts}")
    print(f"Montant à risque : {total_amount:.2f}€")
    print(f"{'='*50}\n")

    context['ti'].xcom_push(key='daily_report', value=report)
    return report

```

```

consolidate = PythonOperator(
    task_id='consolidate_report',
    python_callable=consolidate_report,
)

# =====
# Tâche 4 : Charger le rapport dans BigQuery
# =====
load_report_bq = BigQueryInsertJobOperator(
    task_id='load_report_to_bigquery',
    configuration={
        'query': {
            'query': """
                INSERT INTO `fraudguard-prod.reporting.daily_fraud_summary`
                VALUES (
                    '{{ ds }}',
                    {{ ti.xcom_pull(task_ids='consolidate_report',
key='daily_report')['total_alerts'] }},
                    {{ ti.xcom_pull(task_ids='consolidate_report',
key='daily_report')['total_amount_at_risk'] }},
                    CURRENT_TIMESTAMP()
                )
            """,
            'useLegacySql': False,
        }
    },
    gcp_conn_id='google_cloud_default',
)

# =====
# Tâche 5 : Réentraîner le modèle ML si le taux de faux positifs est trop élevé
# KubernetesPodOperator : lance un pod dédié avec ses propres ressources
# =====
def check_should_retrain(**context):
    """Vérifier si les faux positifs justifient un réentraînement du modèle."""
    all_stats = context['ti'].xcom_pull(task_ids='analyze_alert_type')
    avg_fp_rate = sum(s['false_positive_rate'] for s in all_stats) /
len(all_stats)
    print(f"[FraudGuard] Taux de faux positifs moyen : {avg_fp_rate:.2%}")
    should_retrain = avg_fp_rate > 0.10
    context['ti'].xcom_push(key='should_retrain', value=should_retrain)
    return should_retrain

check_retrain = PythonOperator(
    task_id='check_should_retrain',
    python_callable=check_should_retrain,
)

# Lance un pod Kubernetes avec GPU pour le réentraînement du modèle
retrain_model = GKStartPodOperator(
    task_id='retrain_fraud_model',
    project_id='{{ var.value.gcp_project_id }}',

```



```

        location='europe-west9',
        cluster_name='fraudguard-cluster',
        namespace='fraudguard',
        image='europe-west9-docker.pkg.dev/[PROJECT_ID]/tp2-registry/fraud-ml-
trainer:latest',
        name='fraud-model-retrain-{{ ds_nodash }}',
        arguments=[
            '--training-date', '{{ ds }}',
            '--model-output', 'gs://fraudguard-models/fraud-detector-{{ ds_nodash
}},
        ],
        resources={
            'request_memory': '4Gi',
            'request_cpu': '2',
            'limit_memory': '8Gi',
        },
        get_logs=True,
        # S'exécute seulement si check_should_retrain retourne True
    )

# Ordre des tâches avec Dynamic Task Mapping
fetch_alert_types >> analyze_by_type >> consolidate >> [load_report_bq,
check_retrain]
check_retrain >> _____ # retrain_model

```

Question : Le `KubernetesPodOperator` lance un pod Kubernetes dédié pour le réentraînement ML. Comparez cette approche avec le `PythonOperator` standard pour une tâche de réentraînement qui consomme 8 Go de RAM et 2 GPU. Pourquoi le `KubernetesPodOperator` est-il préférable ?

Réponse :

2.2 — TriggerDagRunOperator : déclencher un DAG depuis un autre

Quand le rapport quotidien détecte un taux de fraude anormalement élevé (> 3× la moyenne hebdomadaire), il doit déclencher immédiatement un DAG d'investigation approfondie — sans attendre le lendemain matin.

Ajoutez dans `days/fraud_daily_report.py` :

```

from airflow.operators.trigger_dagrun import TriggerDagRunOperator

def check_anomalous_day(**context):
    """Détection si ce jour est anormalement frauduleux."""
    report = context['ti'].xcom_pull(task_ids='consolidate_report',
key='daily_report')
    # Seuil : > 500 alertes en une journée = anormal
    is_anomalous = report['total_alerts'] > 500
    context['ti'].xcom_push(key='is_anomalous', value=is_anomalous)
    if is_anomalous:
        print(f"[ALERTE] Jour anormal détecté : {report['total_alerts']} alertes !")
    return is_anomalous

```

```

check_anomaly = PythonOperator(
    task_id='check_anomalous_day',
    python_callable=check_anomalous_day,
)

# Déclencher le DAG d'investigation si la journée est anormale
trigger_investigation = TriggerDagRunOperator(
    task_id='trigger_investigation',
    trigger_dag_id='_____', # 'fraudguard_deep_investigation'
    conf={
        'triggered_by': 'daily_report',
        'trigger_date': '{{ ds }}',
        'alert_count': "{{ ti.xcom_pull(task_ids='consolidate_report',
key='daily_report')['total_alerts'] }}",
    },
    wait_for_completion=False, # Ne pas bloquer le rapport en attendant
    l'investigation
)

# Ajouter à la chaîne de tâches
consolidate >> check_anomaly >> trigger_investigation

```

Partie 3 — Observabilité : Prometheus + Grafana pour Kafka et Airflow

3.1 — Installer la stack de monitoring

```

# Ajouter le repo Helm kube-prometheus-stack
helm repo add prometheus-community https://prometheus-community.github.io/helm-
charts
helm repo update

# Installer Prometheus + Grafana + Alertmanager
helm install monitoring prometheus-community/kube-prometheus-stack \
    --namespace monitoring \
    --create-namespace \
    --set grafana.adminPassword="FraudGuard2026!" \
    --set grafana.service.type=LoadBalancer \
    --timeout 10m

# Attendre que tout soit Running
kubectl get pods -n monitoring -w

# Récupérer l'IP Grafana
GRAFANA_IP=$(kubectl get service monitoring-grafana -n monitoring \
    -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo "Grafana UI : http://${GRAFANA_IP}:80"
echo "Login : admin / FraudGuard2026!"

```

3.2 — Activer les métriques Strimzi dans Prometheus

Créez `kafka/kafka-metrics.yaml` :

```
# PodMonitor : indique à Prometheus de scraper les pods Kafka
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: fraudguard-kafka-metrics
  namespace: fraudguard
  labels:
    release: monitoring # Doit correspondre au label du Prometheus installé
spec:
  selector:
    matchLabels:
      strimzi.io/cluster: fraudguard-kafka
  podMetricsEndpoints:
    - path: /metrics
      port: tcp-prometheus
      interval: _____ # 30s (scrape toutes les 30 secondes)
```

Mettre à jour le cluster Kafka pour exposer les métriques :

```
# Ajouter dans fraudguard-cluster.yaml → spec.kafka
kafka:
  metricsConfig:
    type: jmxPrometheusExporter
    valueFrom:
      configMapKeyRef:
        name: kafka-metrics-config
        key: kafka-metrics-config.yml
```

```
kubectl apply -f kafka/kafka-metrics.yaml

# Vérifier que Prometheus scrape bien les métriques Kafka
kubectl port-forward -n monitoring service/monitoring-kube-prometheus-prometheus
9090:9090 &
# Ouvrir http://localhost:9090
# Query : kafka_server_brokertopicmetrics_messagesin_total
```

3.3 — Dashboard Grafana pour FraudGuard

Dans l'interface Grafana (`http://${GRAFANA_IP}`) :

- Dashboards → New → Import
- Importer le dashboard Kafka officiel Strimzi (ID: 7589)

Créez un dashboard personnalisé FraudGuard avec ces panels :

Panel 1 — Débit de transactions (Messages/s)

```
PromQL : rate(kafka_server_brokertopicmetrics_messagesin_total{topic="transactions-raw"}[1m])
Visualisation : Time series
Seuil d'alerte : > 300 msg/s (pic de fraude potentiel)
```

Panel 2 — Alertes de fraude générées (par type)

```
PromQL : rate(kafka_server_brokertopicmetrics_messagesin_total{topic="fraud-alerts"}[5m]) * 60
Visualisation : Stat + Gauge
```

Panel 3 — Consumer Lag du Fraud Detector

```
PromQL : kafka_consumergroup_lag{consumergroup="fraud-detection-group"}
Visualisation : Time series
Seuil critique : lag > 1000 (le détecteur n'arrive pas à suivre)
```

Panel 4 — Latence de détection (Producer → Alert)

Calculée depuis les logs applicatifs (timestamp tx vs timestamp alerte)
Objectif FraudGuard : < 500ms en P99

Complétez le tableau d'observations après 10 minutes :

Métrique	Valeur observée	Seuil FraudGuard	Status
Transactions/s en régime normal	_____	< 50	_____
Transactions/s pendant l'attaque	_____	Détecté si > 200	_____
Consumer lag fraud-detector	_____	< 100	_____
Latence détection P99	_____ ms	< 500ms	_____
Alertes générées en 10 min	_____	—	—

3.4 — Alerting Grafana : notification en cas d'incident

Dans Grafana (Alerting → Alert Rules → New alert rule) :

```
Rule 1 : Fraud Spike Detection
  Condition : rate(kafka_server_brokertopicmetrics_messagesin_total{topic="fraud-alerts"}[5m]) * 60 > 10
  Duration   : 2 minutes
  Message    : "ALERTE FraudGuard : Plus de 10 alertes/min générées – Incident potentiel"
  Notify     : Email + Slack (configurer dans Contact Points)

Rule 2 : Detector Lag Critical
  Condition : kafka_consumergroup_lag{consumergroup="fraud-detection-group"} > 1000
  Duration   : 5 minutes
```

Message : "CRITIQUE : Le fraud-detector accuse un retard > 1000 messages – Actions en cours non détectées"

Question : Le consumer lag du fraud-detector atteint 5000 messages. Pendant ce temps, l'attaque par micro-transactions de 25 transactions en 30 secondes **n'est pas détectée** car les transactions n'ont pas encore été traitées. Proposez une architecture qui garantirait une latence de détection < 2 secondes même avec un lag important.

Réponse :

Nettoyage Final — IMPORTANT

```
# 1. Supprimer les ressources FraudGuard
kubectl delete -f k8s/fraudguard-deployments.yaml
kubectl delete kafka fraudguard-kafka -n fraudguard
kubectl delete kafkatopics --all -n fraudguard

# 2. Désinstaller la stack monitoring
helm uninstall monitoring --namespace monitoring
kubectl delete namespace monitoring

# 3. Désinstaller Airflow
helm uninstall airflow --namespace airflow
kubectl delete namespace airflow

# 4. Supprimer les namespaces
kubectl delete namespace fraudguard kafka

# 5. Supprimer la base Firestore (alertes de fraude)
gcloud firestore databases delete --database="(default)" --quiet

# 6. Supprimer le cluster GKE
gcloud container clusters delete fraudguard-cluster --region=europe-west9 --quiet

# 7. Vérification
kubectl get namespaces
helm list --all-namespaces
```

Récapitulatif — Compétences validées

- ☐ Kafka Streams : windowing (Tumbling 5 min), état en mémoire, multi-pattern detection
- ☐ Architecture fraude temps réel : Producer → Kafka → Streams → Alerts → Firestore
- ☐ Airflow avancé : Dynamic Task Mapping, KubernetesPodOperator, TriggerDagRunOperator
- ☐ Pipeline hybride : Kafka Streams (temps réel) + Airflow (batch reporting + ML)
- ☐ Observabilité : Prometheus, Grafana, dashboards Kafka, alerting multi-canal

Livrables finaux à remettre

- ☐ Captures d'écran des logs des 3 services en simultané (producer + detector + alert-handler)
- ☐ Au moins 3 alertes de fraude détectées et visibles dans Firestore (screenshot console)
- ☐ DAG `fraudguard_daily_report` avec Dynamic Task Mapping exécuté et graph vert (screenshot)
- ☐ Dashboard Grafana avec les 4 panels complétés (screenshot)
- ☐ Tableau d'observations Kafka Streams rempli
- ☐ `README.md` avec diagramme de l'architecture complète FraudGuard